

CAHIER DES CHARGES

ABSOLUTE STREAM

Plateforme communautaire de suivi de films, séries et animés

Candidat	Thomas MONTOUT
Épreuve	E6 – BTS SIO option SLAM
Projet	Absolute Stream
Binôme	Alexis DE JESUS
Stack technique	TypeScript · React 19 · Next.js 16 · PostgreSQL (NeonDB)
Année	2026

SOMMAIRE

- I. Présentation du Projet
- II. Analyse des Besoins et Périmètre
- III. Description Graphique et Ergonomique
- IV. Architecture Technique
- V. Modèle Conceptuel et Dictionnaire de Données
- VI. Description Fonctionnelle et Documentation API
- VII. Gestion de Session et Sécurité
- VIII. Tests et Validation
- IX. Mes Réalisations Personnelles
- X. Bilan et Perspectives
- XI. Annexes

I. Présentation du Projet

1. Contexte

Dans le cadre de l'épreuve E6 du BTS SIO option SLAM, j'ai développé le projet Absolute Stream en binôme avec Alexis DE JESUS. Ce projet vise à valider notre maîtrise de la conception, du développement et du déploiement d'une application web full-stack en conditions réelles, en s'appuyant sur une stack moderne : TypeScript, React 19 et Next.js 16.

2. La Vision et les Objectifs

Absolute Stream est une plateforme communautaire inspirée de Letterboxd, unifiant films, séries et animés en une seule interface. L'objectif central est de résoudre la fragmentation des historiques de visionnage éparpillés sur de nombreuses plateformes (Netflix, Disney+, Crunchyroll...).

Les objectifs principaux du projet sont :

- Centraliser le suivi des œuvres vues, en cours et à voir
- Proposer une dimension sociale (abonnements, critiques, partage de listes)
- Garantir une expérience fluide, rapide et accessible (Mobile First, Dark Mode)
- Anticiper des fonctionnalités futures : système de Match en duo et tournois communautaires

3. Périmètre du MVP présenté

Pour la soutenance, nous avons fait le choix délibéré de présenter un Produit Minimum Viable (MVP) fonctionnel et stable, centré sur l'exploration de médias. Une grande partie de notre travail de conception a consisté à modéliser la base de données de l'application finale (Match, réseau social, critiques, listes). Cependant, en termes d'interface utilisateur (UI) livrable aujourd'hui, le périmètre assumé couvre exclusivement les piliers fondamentaux : la consultation avancée du catalogue (API TMDb), la recherche globale, les fiches détaillées riches, l'authentification sécurisée et les fondations du profil utilisateur.

II. Analyse des Besoins et Périmètre

1. Acteurs du système

Acteur	Description
Visiteur	Utilisateur non authentifié. Peut naviguer librement dans le catalogue, utiliser la recherche globale et consulter les fiches détaillées des œuvres.
Membre	Utilisateur authentifié. Dispose d'un compte sécurisé et d'un profil public posant les bases des futures interactions sociales.
API TMDB	Fournisseur de données externe (The Movie Database). Sollicité dynamiquement pour alimenter l'ensemble du catalogue d'œuvres.

2. Besoins fonctionnels (MVP)

- ▶ **Catalogue Unifié** : Navigation par carrousels, pages de découverte avec filtres dynamiques (genres/tris), page Collections, et isolation technique automatique des animés.
- ▶ **Recherche Globale Avancée** : Moteur de recherche centralisé avec menu déroulant, optimisé pour les performances réseau.
- ▶ **Fiches Détaillées Enrichies** : Affichage immersif regroupant les métadonnées localisées (dates FR, durées), le casting principal et l'intégration automatisée de la bande-annonce YouTube officielle.
- ▶ **Authentification et Profil** : Inscription et connexion sécurisées via *Better Auth*, persistance de la session et page de profil public en lecture seule.

3. Besoins non-fonctionnels

- ▶ **Ergonomie** : interface Dark Mode, responsive Mobile First, navigation fluide sur tous supports
- ▶ **Performances** : rendu SSR via le routeur Next.js pour optimiser le temps de chargement et le SEO
- ▶ **Sécurité** : protection des clés API, sessions sécurisées, protection contre les injections SQL et les failles XSS
- ▶ **Maintenabilité** : typage strict TypeScript, architecture modulaire, documentation continue

III. Description Graphique et Ergonomique

1. Thème et Design

L'interface adopte un thème sombre (Dark Mode) avec des arrière-plans dynamiques qui s'adaptent à l'affiche du média consulté, grâce à un hook personnalisé `useImageColor`. Cette approche crée une immersion visuelle forte, donnant l'impression que l'interface « vit » en fonction du contenu affiché.

L'ensemble des styles est géré via Tailwind CSS v4, permettant une cohérence visuelle et une maintenance simplifiée.

2. Composants UI et Accessibilité

- ▶ **Radix UI** : primitives accessibles pour les menus déroulants, sliders et navigation au clavier
- ▶ **Embla Carousel** : défilement horizontal fluide pour les listes de films et séries
- ▶ **shadcn/ui** : composants prêts à l'emploi cohérents avec le design system

3. Principales interfaces développées

Interfaces dont j'ai assuré le développement principal

- Pages de catalogue interactives avec logique de filtrage dynamique
- Barre de recherche globale interactive (Menu déroulant dynamique avec gestion du *debounce*)
- Vues spécifiques de la fiche média (`InformationView`, `CastingView`) et intégration du lecteur vidéo
- Composants d'affichage en carrousel (`EmblaCarousel`, `MediaCards`)
- Page spécifique aux Animés (`/animés`) avec isolation technique des flux
- Refonte de la page d'accueil (Landing Page) et intégration de l'identité visuelle (Logo)
- Layout global de bas de page (`Footer.tsx`)

IV. Architecture Technique

Le projet exploite l'architecture Serverless de Next.js, qui permet au code client (Frontend) et à la logique serveur (Backend) de cohabiter de manière sécurisée et maintenable au sein d'un même repository.

Stack Technologique

Technologie		Rôle dans le projet
Next.js 16	Framework principal	App Router, Server Actions, SSR, Middleware de protection des routes
React 19	Bibliothèque UI	Composants fonctionnels, hooks, gestion d'état (Context API)
TypeScript 5	Typage statique	Sécurisation du code, interfaces pour les objets TMDB, détection des erreurs à la compilation
NeonDB	Base de données	PostgreSQL hébergé dans le cloud, scalable et sans maintenance serveur
Prisma 6	ORM	Requêtes SQL sécurisées et typées, schéma déclaratif, migrations automatisées
Better Auth	Authentification	Hachage des mots de passe, sessions via cookies HTTP-Only
Tailwind CSS v4	Styles	Classes utilitaires, thème sombre, responsive design
Radix UI	Composants	Accessibilité ARIA, primitives headless pour menus et interactions

Schéma de l'Architecture

FRONTEND (Client)	BACKEND (Serveur)
– Composants React / Next.js – Tailwind CSS + Radix UI – AuthContext global – Optimistic UI (favoris)	– Route Handlers (proxy TMDB) – Server Actions (mutations BDD) – Prisma ORM + NeonDB – Better Auth (sessions)

V. Modèle Conceptuel et Dictionnaire de Données

Le catalogue d'œuvres n'est pas stocké dans notre base de données – il est géré dynamiquement via l'API TMDB. Notre base PostgreSQL se concentre exclusivement sur les entités liées aux utilisateurs et à leurs interactions.

Principales entités du schéma Prisma

Entité	Champs principaux	Description
User	id, name, email, image	Compte utilisateur, géré par Better Auth
Library	userId, tmdbId, type, status	Statut de visionnage unique par utilisateur et par œuvre (Vu / À voir / En cours)
Review	userId, tmdbId, rating?, content?, visibility	Critique avec note et texte optionnels. Visibilité : PUBLIC ou FRIENDS
Favorite	userId, tmdbId, mediaType	Mise en favoris d'une œuvre
List	userId, name, isPublic	Liste personnalisée publique ou privée
ListItem	listId, tmdbId, mediaType	Entrée dans une liste personnalisée
Follow	followerId, followingId	Relation d'abonnement entre deux utilisateurs
<i>MatchSession*</i>	id, userId1, userId2, status	Anticipé : session de match en duo (non activé dans le MVP)

* Afin de préparer l'évolution future du projet, l'ensemble du modèle relationnel a été implémenté (incluant Library, Review, List, Follow et le système de Match). Cependant, dans le périmètre strict du MVP présenté, ces entités ne sont pas encore reliées à des interfaces graphiques utilisateur.

VI. Description Fonctionnelle et Documentation API

Pour garantir la sécurité de la clé `TMDB_API_KEY`, le navigateur ne communique jamais directement avec TMDB. J'ai mis en place une architecture où notre serveur joue le rôle de proxy.

1. Route Handlers – Proxy TMDB (Lecture)

L'intégralité des flux de données externes a été sécurisée via nos propres routes API.

Endpoints TMDB implémentés

Films (`/api/movies/`)

- GET `/api/movies/discoverMovies` Liste filtrée et paginée (params : page, genre)
- GET `/api/movies/popularMovies` Films populaires du moment
- GET `/api/movies/topRated` Films les mieux notés par la communauté TMDB
- GET `/api/movies/nowPlaying` Films actuellement à l'affiche au cinéma
- GET `/api/movies/movieGenres` Référentiel des identifiants de genres de films

Séries (`/api/tvshows/`)

- GET `/api/tvshows/discoverTvshows` Séries filtrées et paginées
- GET `/api/tvshows/popularTv` Séries populaires
- GET `/api/tvshows/topRated` Séries les mieux notées
- GET `/api/tvshows/onTheAir` Séries en cours de diffusion (nouveaux épisodes)
- GET `/api/tvshows/tvGenres` Référentiel des identifiants de genres de séries

Animés (`/api/animés/`)

- GET `/api/animés/discoverAnimes` Catalogue d'animés (avec isolation technique des flux)
- GET `/api/animés/popularAnimes` Animés populaires du moment
- GET `/api/animés/topRated` Animés les mieux notés
- GET `/api/animés/animeGenres` Référentiel des genres d'animation japonaise

Global & Recherche

- GET `/api/search` Moteur de recherche global (`/search/multi`)
- GET `/api/collections` Recherche de sagas et univers cinématographiques
- GET `/api/findByID/[external_id]` Lookup universel d'une œuvre via un ID externe

2. Server Actions – Mutations en écriture

Les actions modifiant la base de données utilisent les Server Actions Next.js. Elles s'exécutent côté serveur, vérifient la session via Better Auth, puis effectuent la requête Prisma appropriée.

Action	Table concernée	Comportement
toggleFavorite(mediaId, mediaType)	Favorite	Crée ou supprime un favori, UI mise à jour en Optimistic UI

VII. Gestion de Session et Sécurité

La sécurité a été un axe central de mon développement, notamment sur les aspects authentification et gestion de session.

1. Authentification avec Better Auth

- Hachage des mots de passe via l'algorithme scrypt (algorithme par défaut de Better Auth), conçu pour être lent et gourmand en mémoire afin de résister aux attaques par force brute.
- Cookies de session en HTTP-Only : en cas de faille XSS, le token de session reste illisible par JavaScript et ne peut donc pas être volé.
- Protection native contre les failles XSS par l'échappement automatique des variables au rendu par React (JSX), qui neutralise l'injection de balises.
- Gestion propre de l'état de session côté client via un AuthContext global

2. Protection des Routes (Middleware Next.js)

Un fichier middleware intercepte toutes les requêtes réseau pour s'assurer que seuls les membres authentifiés accèdent aux pages privées.

3. Protection contre les Injections SQL

Prisma génère des requêtes paramétrées : les valeurs saisies par l'utilisateur sont transmises séparément de la syntaxe SQL, ce qui rend l'injection SQL structurellement impossible.

4. Sécurité de l'API TMDB

La clé TMDB_API_KEY est stockée comme variable d'environnement serveur et n'est jamais exposée côté client. Le proxy via les Route Handlers garantit que le navigateur ne peut pas accéder directement à cette clé.

Sécurisation et Typage des flux externes (contribution spécifique)

- L'API TMDB manipulant des volumes de données massifs et parfois imprévisibles, j'ai mis en place un système de typage strict (*TypeScript Interfaces*).

VIII. Tests et Validation

1. Intégration Continue (CI/CD)

Un workflow GitHub Actions (ci.yml) a été mis en place pour garantir la stabilité du projet à chaque push. Ce pipeline automatique :

- Lance ESLint pour vérifier la conformité du code aux règles définies
- Exécute tsc --noEmit pour détecter toutes les erreurs de typage TypeScript
- Bloque les merges si l'une des vérifications échoue

2. Tests Fonctionnels Manuels

Les parcours utilisateurs critiques ont été validés manuellement :

Scénario testé	Résultat attendu	Statut
Inscription d'un nouvel utilisateur	Compte créé, session ouverte, redirection profil	Validé
Connexion / Déconnexion	Session persistante, NavBar cohérente	Validé
Ajout en favori (Optimistic UI)	UI mise à jour immédiatement, BDD persistée	Validé
Navigation vers une page privée sans session	Redirection vers /login par le Middleware	Validé
Suivi d'un autre utilisateur	Compteurs Followers/Following mis à jour	Indisponible
Création d'une liste privée	Liste visible uniquement pour l'auteur	Indisponible
Recherche globale d'un film	Affichage dynamique dans le menu déroulant	Validé

IX. Mes Réalisations Personnelles

Au sein du binôme, j'ai piloté l'intégration des données métiers (API TMDB), la logique de filtrage du catalogue, la documentation technique et l'infrastructure d'affichage. Ces travaux représentent plus d'une vingtaine de commits majeurs sur le repository.

1. Backend et Proxy API (TMDB)

La récupération des données est le cœur fonctionnel du catalogue :

- **Création des Route Handlers** : Implémentation complète des routes API sécurisées `/api/movies`, `/api/tvshows` et `/api/animes` (endpoints `discover`, `popular`, `topRated`, `genres`, etc.).
- **Typage Strict** : Création des interfaces et types TypeScript (`src/types/tmdb.ts`) pour mapper les réponses complexes de l'API TMDB de manière sécurisée.
- **Moteur de Recherche Global** : Développement de la route `/api/search/route.ts` branchée sur le point d'entrée `/search/multi` de TMDB, centralisant la recherche de tous les médias.

2. Logique de Catalogue et Filtrage

- ▶ **Algorithmes de filtres** : Mise en place, dédoublonnage et factorisation de la logique de filtrage dynamique (par genres, par tri) sur les catégories Films, Séries et Animés.
- ▶ **Séparation technique** : Conception de la page dédiée aux Animés et implémentation de la logique d'exclusion empêchant les animés de polluer les résultats des séries classiques.

3. Composants UI, Affichage et Carrousels

- ▶ **Navigation visuelle** : Intégration avancée de la bibliothèque *Embla Carousel* via la création du composant réutilisable `MediaCards`, optimisant l'affichage en rangées.
- ▶ **Layout global** : Refonte structurelle de la *Home page*, intégration du logo applicatif, et création du `Footer.tsx`.
- ▶ **Micro-interactions** : Améliorations de l'UX (ajout des curseurs d'action sur le carrousel, renommage stratégique des menus).
- ▶ **Barre de Recherche Interactive** : Refactoring du composant `Search.tsx` en *Client Component* intégrant des concepts avancés d'optimisation : utilisation d'un délai d'attente (`debounce` de 300ms) pour limiter les appels API lors de la frappe, et implémentation d'un `AbortController` pour annuler les requêtes obsolètes et éviter les conflits d'affichage. Le menu déroulant filtre automatiquement les profils d'acteurs pour se concentrer sur les œuvres.

4. Optimisation des Flux et Traitement des Données

- ▶ **Réduction des requêtes HTTP** : Optimisation des appels sur la page unifiée `/view-media/[type]/[id]` en utilisant le paramètre `append_to_response` (récupération simultanée des crédits, vidéos et dates de sortie en une seule requête).
- ▶ **Normalisation des données** : Développement de *helpers* (`InformationView.tsx`) pour uniformiser l'affichage entre les films et les séries (gestion conditionnelle des réalisateurs/créateurs, formatage croisé de la durée `runtime` vs `episode_run_time`, et conversion des dates/classifications au format français)
- ▶ **Algorithme de sélection vidéo** : Création du helper `findTrailerKey` qui priorise intelligemment la récupération de la bande-annonce officielle YouTube via TMDB, avec un système de *fallback* (recherche de secours) en cas d'absence de données.

5. Documentation et Infrastructure Pédagogique

J'ai pris en charge la documentation et le transfert de compétences du projet :

- ▶ **Rédaction métier** : Mise à jour exhaustive du `README.md` (alignement sur la structure finale).
- ▶ **Création de contenu** : Rédaction d'un référentiel de cours complet de plus de 8 300 lignes, réparti sur 18 fichiers (`docs/cours/`). Ce guide documente l'apprentissage de A à Z de notre stack (Next.js, Prisma, Better Auth, Tailwind).
- ▶ **Maintenance de l'Infra** : Résolution des erreurs bloquantes sur la CI (GitHub Actions) et ajout de commentaires *inline* dans le code pour la soutenance.

Résumé quantitatif de mes contributions

- Plus d'une vingtaine de commits majeurs sur le repository GitHub.
- Développement intégral de la couche Backend/API (Proxy TMDB) avec plus de 15 endpoints sécurisés et un moteur de recherche global.
- Optimisation des performances réseau et client (Debounce, AbortController, requêtes TMDB groupées via `append_to_response`).
- Création d'un système de typage strict (TypeScript) pour l'assainissement des données externes.
- Conception des algorithmes de filtrage dynamique (catégories, genres, tris) et normalisation logicielle des flux Séries/Films/Animés.
- Développement des interfaces d'exploration tactiles (intégration d'Embla Carousel et des MediaCards)
- Création d'une infrastructure documentaire exhaustive, incluant un référentiel de cours technique de plus de 8 300 lignes.

X. Bilan et Perspectives

1. Compétences validées

Le développement d'Absolute Stream m'a permis de valider des compétences concrètes de développeur Full-Stack dans un contexte réel et exigeant :

- Maîtrise de l'architecture Next.js hybride (Route Handlers, Server Actions, Client/Server Components).
- Intégration avancée et sécurisation d'API tierces (Création de proxy interne pour TMDB, réduction des requêtes HTTP).
- Optimisation des performances Front-End
- Sécurisation des flux de données avec des interfaces TypeScript strictes.
- Développement d'interfaces dynamiques et accessibles avec React 19, Radix UI et Embla Carousel.

- Rédaction de documentation technique professionnelle.
- Mise en place d'un pipeline CI avec GitHub Actions (lint + typage TypeScript)
- Travail collaboratif en binôme avec Git (branches, code review, gestion des conflits)

2. Difficultés rencontrées et solutions apportées

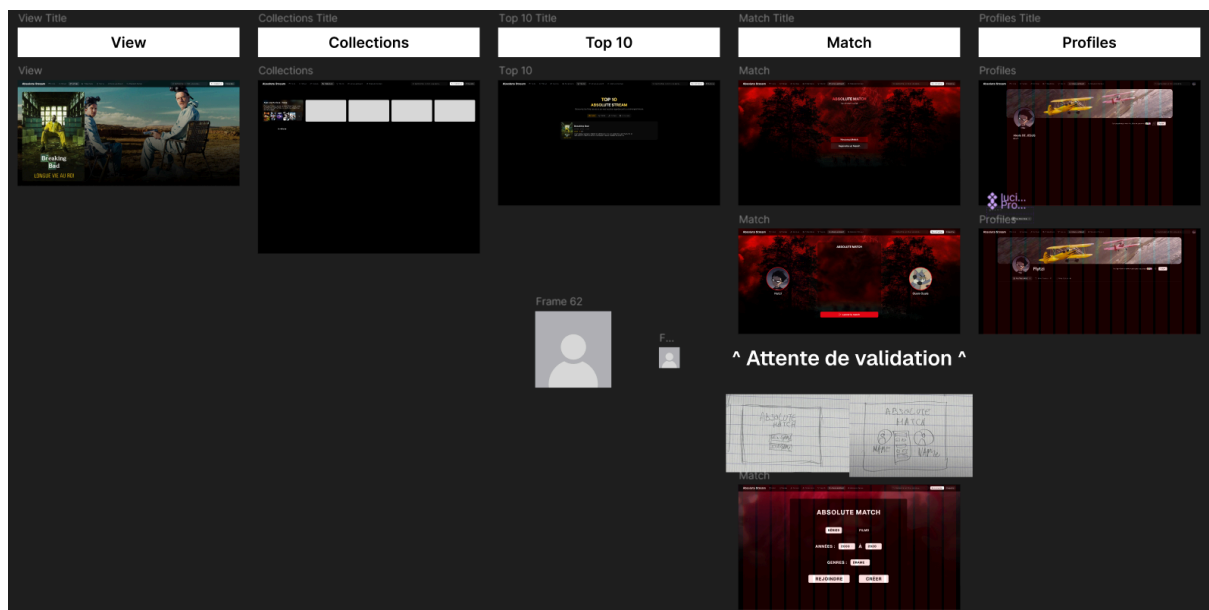
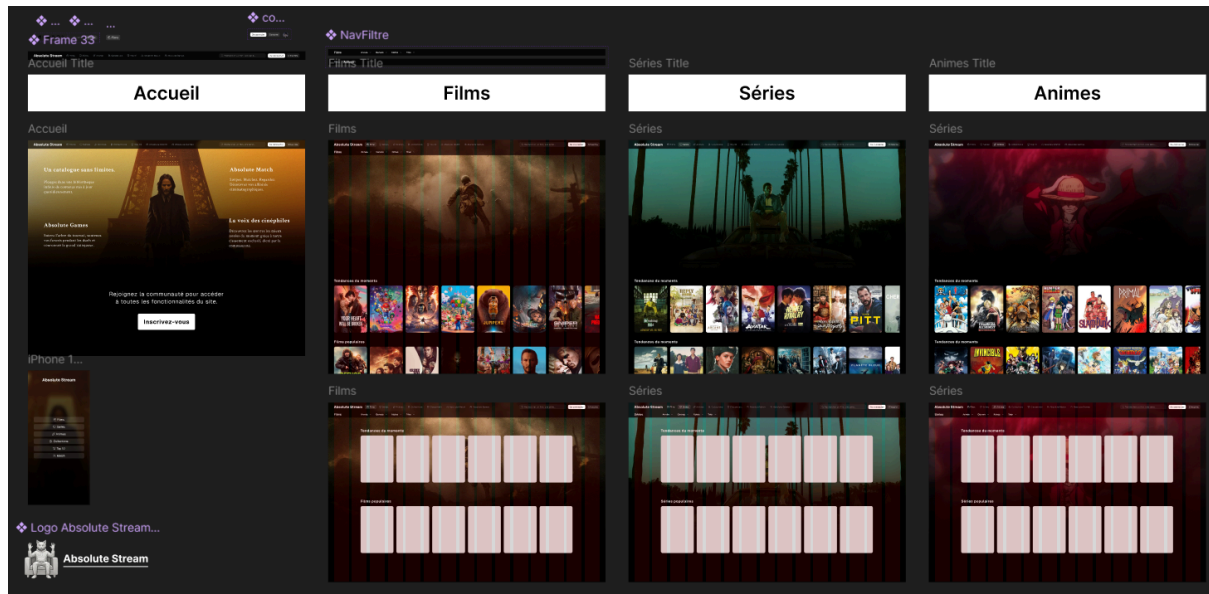
Difficulté	Solution apportée
Typage de données JSON massives et imprévisibles	Création d'interfaces TypeScript strictes (<i>TmdbMedia</i>) pour assainir les payloads avant traitement par les composants React.
Isolation des flux "Animés" et "Séries"	Implémentation d'une logique de filtrage ciblé par paramètre (ID de genre) lors du <i>data-fetching</i> côté serveur.
Gestion fluide du défilement des médias	Intégration et paramétrage de la bibliothèque <i>Embla Carousel</i> pour garantir des performances optimales sur mobile et desktop.

3. Perspectives d'évolution

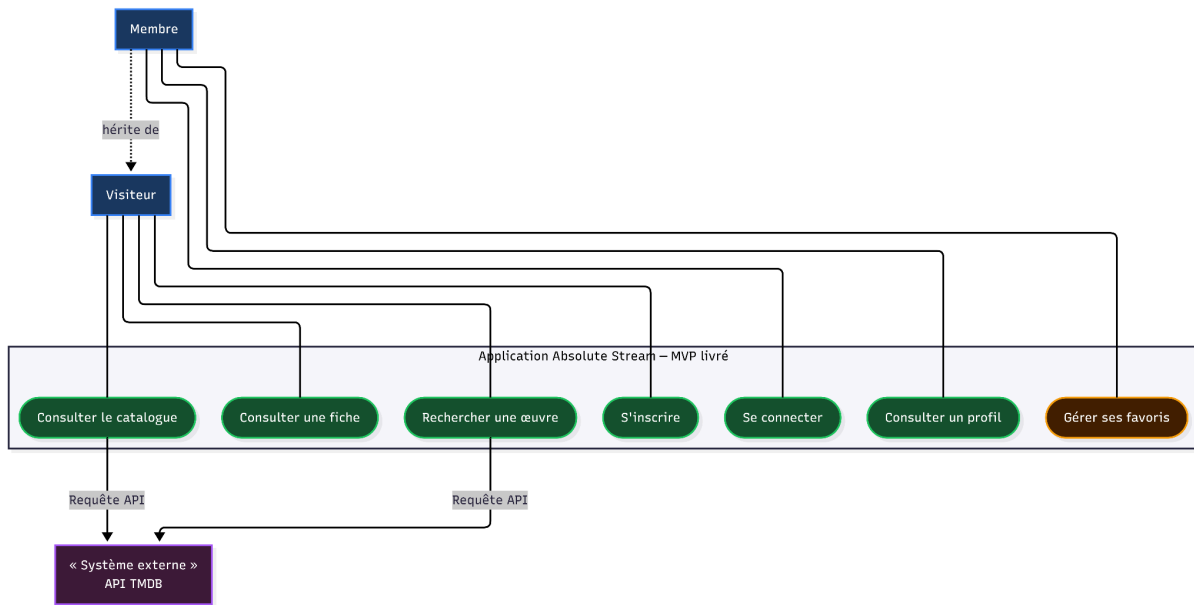
- ▶ **Système de Match** : développer l'interface de Swipe (les tables BDD sont déjà prêtes) pour activer la fonctionnalité phare
- ▶ **Application Mobile** : exploiter la séparation de notre logique Backend pour développer une app React Native
- ▶ **Recommandations personnalisées** : algorithme basé sur l'historique de visionnage de l'utilisateur
- ▶ **Notifications en temps réel** : WebSockets pour les interactions sociales (nouveau follower, réponse à une critique)

XI. Annexes

Annexe 1 : Maquettes et Ergonomie (UI/UX)

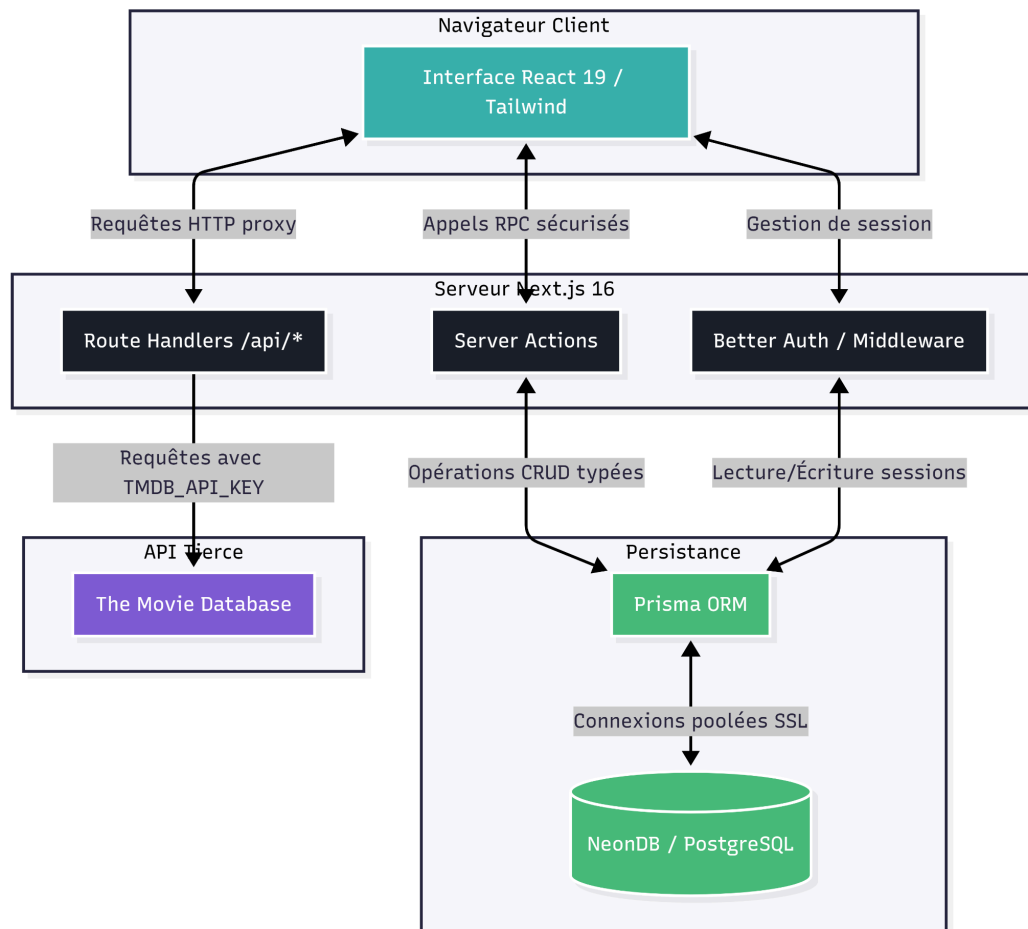


Légende : Le design met en évidence le choix du thème sombre (*Dark Mode*) et l'utilisation d'effets de transparence pour valoriser les affiches issues de l'API TMDB

Annexe 2 : Diagramme de cas d'utilisation (périmètre MVP livré).

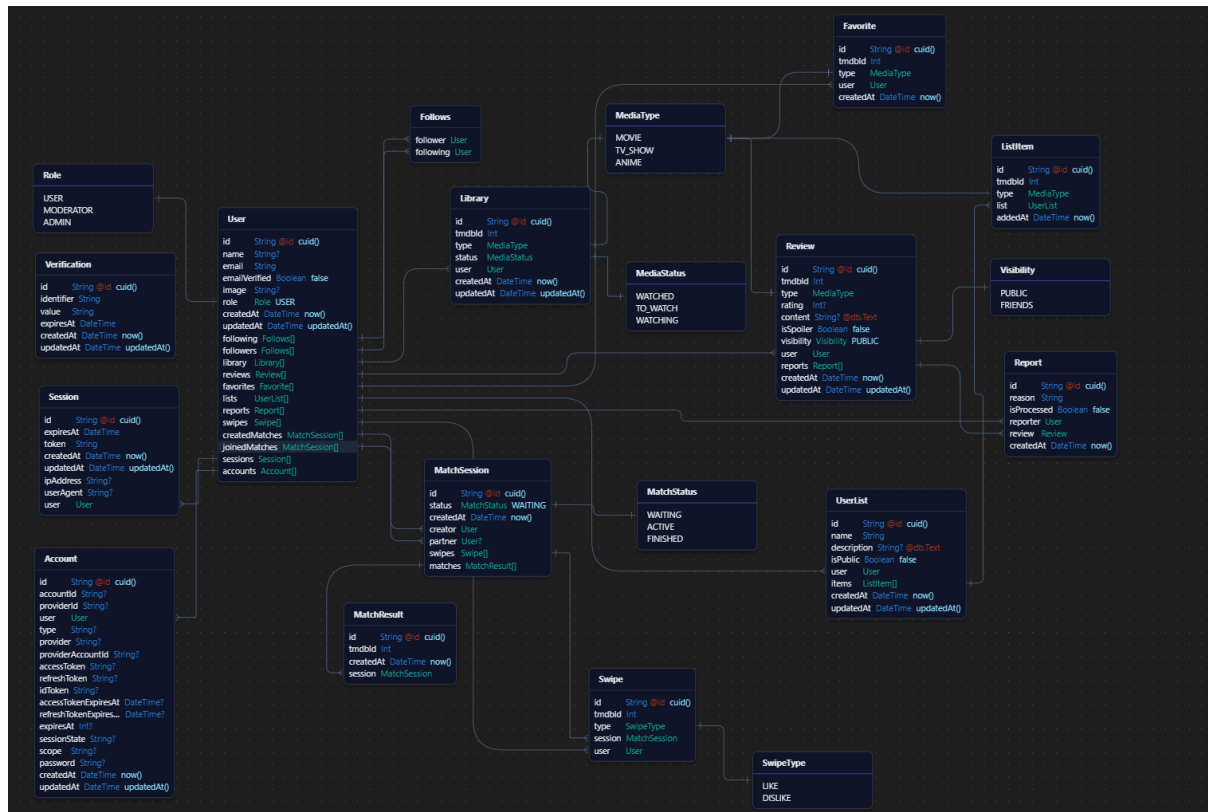
Légende : Interactions entre les acteurs accessibles dans le MVP (Visiteur, Membre) et l'API externe TMDB. Couvre la consultation du catalogue, les fiches détaillées, la recherche, l'authentification, le profil et la gestion des favoris (partiellement validée). La généralisation traduit l'héritage des droits du Membre vis-à-vis du Visiteur.

Annexe 3 : Architecture Technique (Serverless)



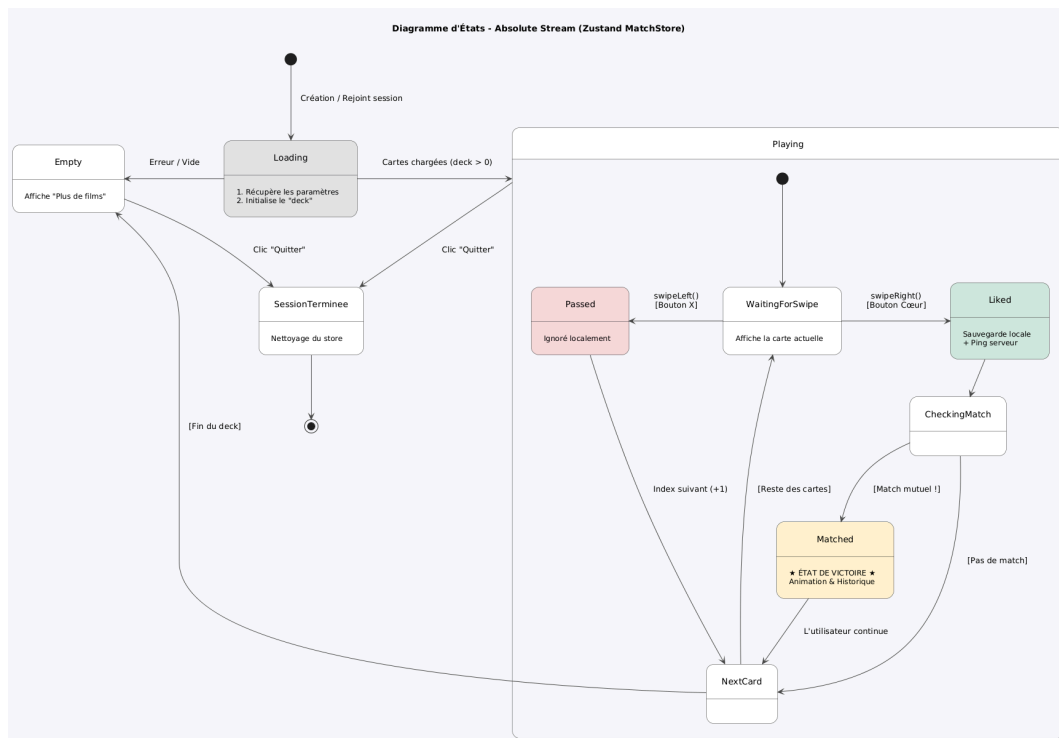
Légende : Schéma global de l'architecture logicielle de la plateforme. Il illustre le flux de données entre le navigateur client (Frontend React), le serveur Next.js (Route Handlers et Server Actions agissant comme proxy), la base de données PostgreSQL gérée via Prisma (NeonDB), et l'API externe The Movie Database.

Annexe 4 : Schéma Prisma



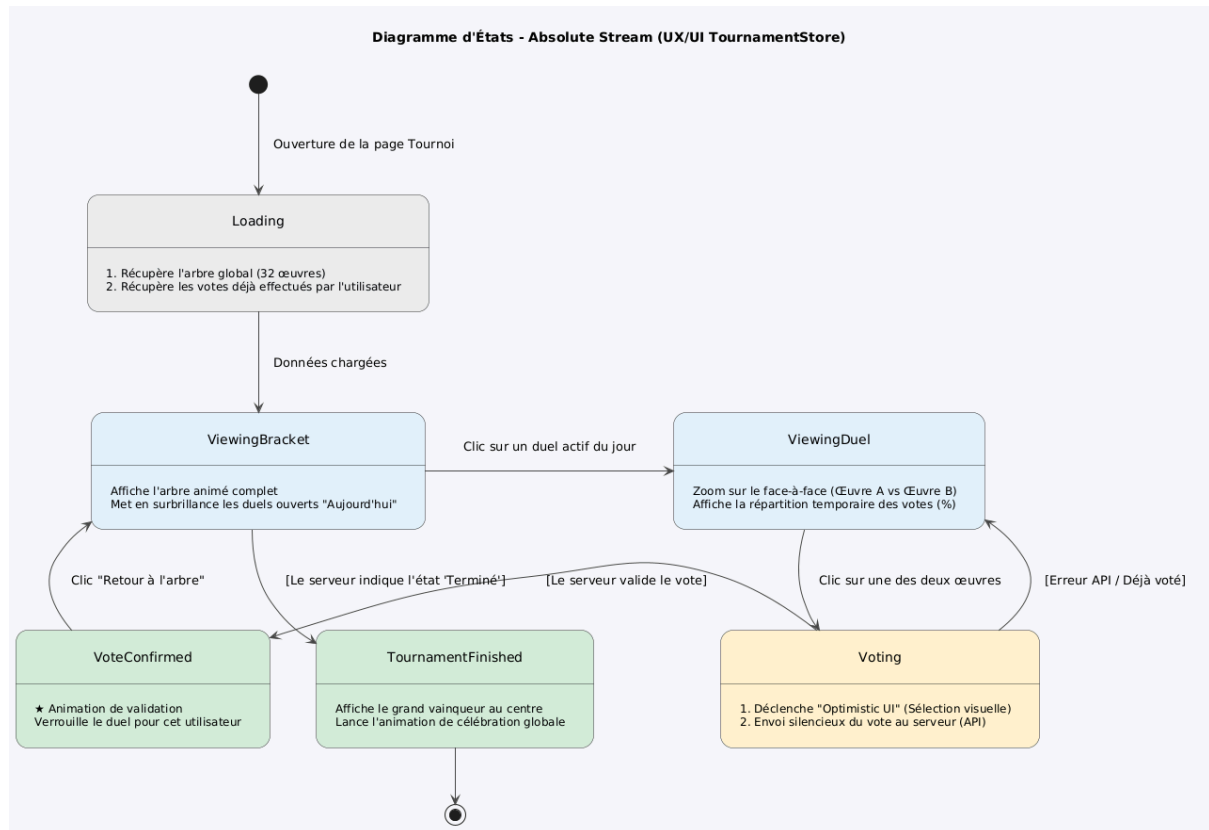
Légende : Représentation du schéma relationnel de la base de données. Ce modèle met en évidence la séparation entre les entités fonctionnelles du MVP (User, Library, Review, Follows) et l'anticipation architecturale des futures itérations (tables MatchSession, Swipe, MatchResult), démontrant la scalabilité du projet.

Annexe 5 : Diagramme d'États-Transitions - Session de Match



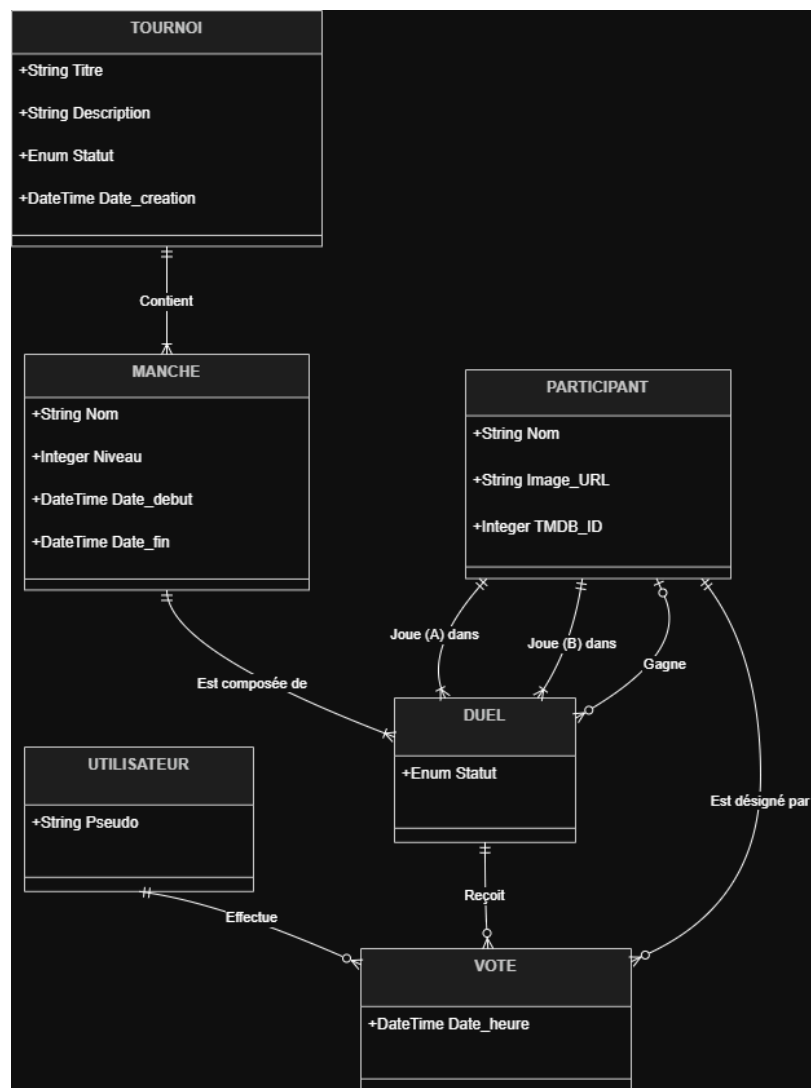
Légende : Modélisation du cycle de vie d'une session de "Match" en duo. Ce diagramme d'états-transitions illustre les différents états d'une session et les événements déclencheurs permettant de passer d'un état à l'autre.

Annexe 6 : Diagramme d'États-Transitions - Tournoi Communautaire



Légende : Modélisation du cycle de vie d'un Tournoi. Ce schéma détaille les phases successives de cet événement communautaire.

Annexe 7 : Modèle Conceptuel de Données (MCD) - Tournoi



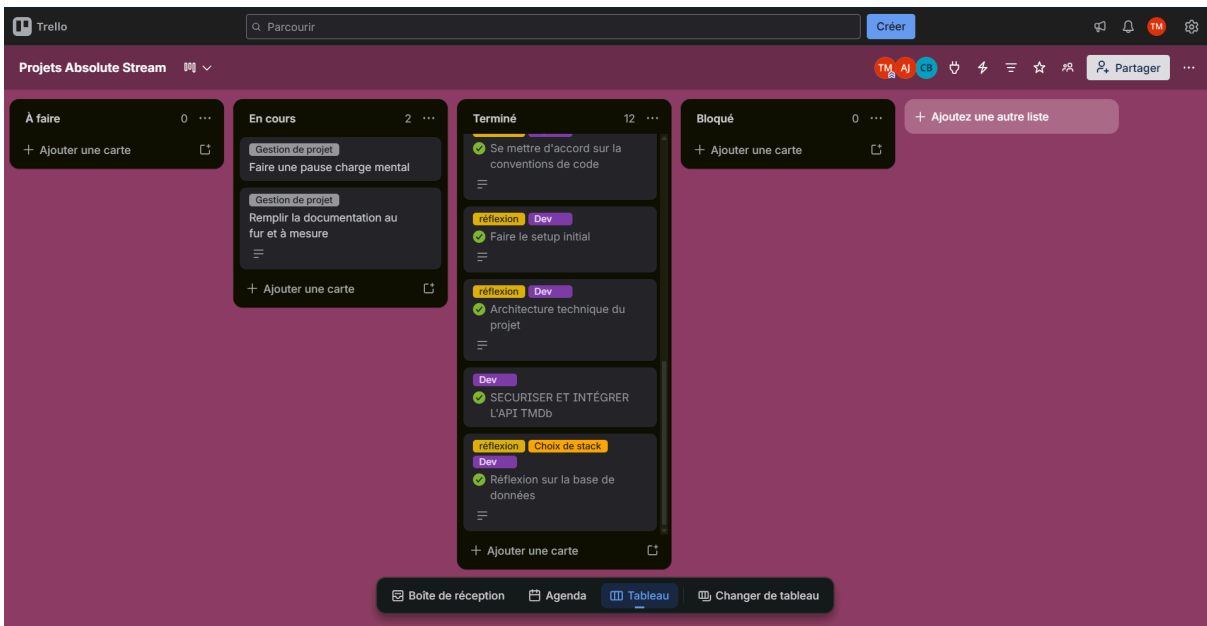
Légende : Ce modèle met en évidence la structure relationnelle nécessaire pour stocker les confrontations, comptabiliser les votes des membres et gérer l'arbre de progression, confirmant la scalabilité de notre architecture de données actuelle.

Annexe 8 : Dictionnaire de Données - Tournoi

Entité logique	Nom de la donnée (Attribut)	Type	Description / Exemple	Page tournoi admin	
TOURNOI	Titre du tournoi	Texte	Ex: "Le meilleur Anime de la décennie"	Nom	Description
TOURNOI	Description du tournoi	Texte	Ex: "Votez chaque jour pour élire..."	Bouton Démarrer un tournoi	A la fonction de démarrer un tournoi (ouvrir la page config tournoi)
TOURNOI	Statut du tournoi	Énumération	Ex: "À venir", "En cours", "Terminé"	Bouton Terminer un tournoi	A la fonction de terminer le tournoi terminer ou en cours
TOURNOI	Date de création	Date/Heure	La date où l'admin a créé le tournoi	Button Historique des tournoi	A la fonction d'ouvrir la page historique des tournoi pour afficher
MANCHE	Nom de la manche	Texte	Ex: "16ème de finale", "Quart de finale"	Page config tournoi admin	
MANCHE	Niveau de la manche	Nombre entier	Ex: 32, 16, 8, 4, 2 (sert pour l'affichage de l'arbre)	Nom	type
MANCHE	Date de début	Date/Heure	Moment où les votes ouvrent pour cette manche	Type Mode	description
MANCHE	Date de fin	Date/Heure	Moment où les votes ferment (pour le compte à rebours)	selector	Choisir le type de tournoi (vote général / choix imposé)
DUEL	Statut du duel	Énumération	Ex: "En attente", "En cours", "Terminé"	number or selector	Choisir le temps de vote par manche ou par duel - ou genere automatiquement pas la d
PARTICIPANT	Nom de l'oeuvre/sujet	Texte	Ex: "Nanito", "L'Attaque des Titans"	number or selector	Choisir la date de fin - ou genere automatiquement par les temps de vote
PARTICIPANT	Image d'illustration	Texte (URL)	Le lien vers l'affiche		
PARTICIPANT	Identifiant TMDB	Nombre entier	(Optionnel) Pour faire le lien avec votre base de films		
UTILISATEUR	Pseudo	Texte	Ex: "Thomas" (Les autres infos sont gérées par l'Auth)		
VOTE	Date et heure du vote	Date/Heure	Pour savoir exactement quand le joueur a cliqué		

Légende : Extrait du dictionnaire de données définissant les entités liées au système de Tournoi. Ce document de conception précise le typage strict, les formats et les contraintes d'intégrité de chaque champ. Il garantit une future implémentation rigoureuse et sécurisée au sein de notre base de données PostgreSQL.

Annexe 9 : Gestion de Projet et Planification



Légende : Extrait du tableau de suivi (Trello) illustrant la méthodologie adoptée. Cet outil a permis de prioriser de structurer les sprints de développement et de répartir efficacement les tâches.

Annexe 10 : Extrait de Code

The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'src', 'components', 'lib', and 'providers'. The code editor displays the file 'Search.tsx' with the following code:

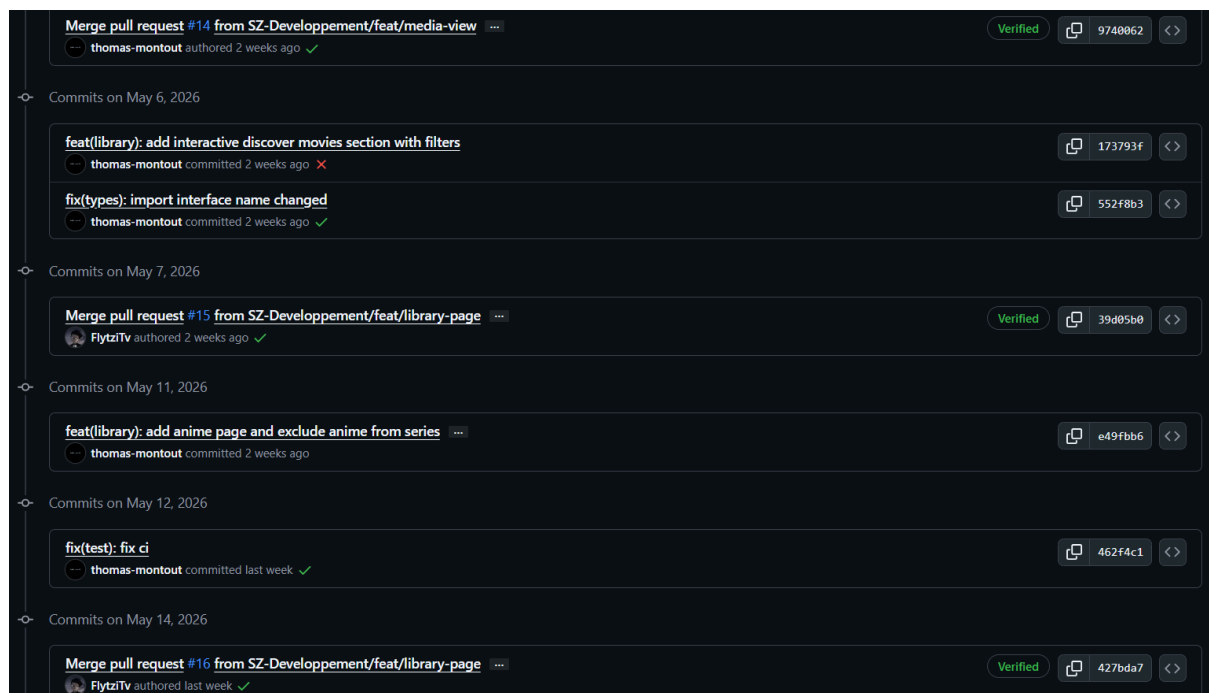
```

13  export default function InputSearch({ className }: { className?: string }) {
14
15    // ----- Fetch deboncé -----
16    // À chaque frappe on déclenche un setTimeout. Si l'utilisateur retape avant 300ms,
17    // on annule l'ancien timer → un seul appel réseau pour une rafale de touches.
18    useEffect(() => {
19      const trimmed = query.trim();
20      if (!trimmed) {
21        setResults([]);
22        setLoading(false);
23        return;
24      }
25      setLoading(true);
26      // AbortController pour annuler une requête en vol si l'utilisateur retape :
27      // évite un cas où une vieille réponse arrive APRÈS la nouvelle et écrase
28      // l'état avec des résultats périmés.
29      const controller = new AbortController();
30      const timer = setTimeout(async () => {
31        try {
32          const res = await fetch(
33            `/api/search?q=${encodeURIComponent(trimmed)}`,
34            { signal: controller.signal },
35          );
36          if (!res.ok) {
37            setResults([]);
38            return;
39          }
40          // You, il y a 5 heures • feat: add TMDB multi-search bar with debounced ...
41          const data = await res.json();
42          // On filtre les personnes pour ne garder que films/séries.
43          const filtered: MultiSearchResult[] = (data.results ?? []).filter(
44            (r: MultiSearchResult) =>
45              r.media_type === "movie" || r.media_type === "tv",
46          );
47          setResults(filtered.slice(0, MAX_RESULTS));
48        } catch (err) {
49          if ((err as Error).name !== "AbortError") {
50            console.error("Search error:", err);
51            setResults([]);
52          }
53        } finally {
54          setLoading(false);
55        }
56      }, DEBOUNCE_MS);
57    }, [query]);
58  }

```

Légende : Extrait du code source illustrant la logique métier de la barre de recherche globale. Il met en évidence l'optimisation des performances Front-End via l'implémentation d'un délai d'attente pour limiter le nombre de requêtes HTTP lors de la frappe.

Annexe 11 : Historique de Versionnement et Travail Collaboratif (Git)



Légende : Capture de l'historique des validations (*commits*) illustrant notre gestion de projet sous Git. Cet extrait met en évidence la méthodologie de travail collaboratif, de régularité des intégrations de code, clarté et normalisation des messages de *commit* illustrant les responsabilités de chacun, et utilisation de branches pour isoler les développements fonctionnels.

Thomas MONTOUT – BTS SIO option SLAM – 2026